

Name: [Your Name]

Lab Program Number: 11

Date: [Submission Date]

Title: Understanding and Implementing 2D Scaling

Theory

2D Scaling

2D Scaling is a geometric transformation that changes the size of an object by scaling its coordinates relative to a fixed point, usually the origin. Scaling can be uniform (same factor in both directions) or non-uniform (different factors in x and y directions).

Mathematical Representation:

For a point (x,y) scaled by scaling factors s_x and s_y about the origin, the new coordinates (x',y') are:

$$\begin{aligned}x' &= x \cdot s_x \\ y' &= y \cdot s_y\end{aligned}$$

Matrix Representation:

In homogeneous coordinates, scaling about the origin can be represented using a 3×3 transformation matrix:

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x & y & 1 \end{bmatrix}$$

Scaling About an Arbitrary Point:

If scaling is about a fixed point (x_f, y_f) , the transformation includes:

1. Translate point so that (x_f, y_f) is at origin
2. Scale by s_x and s_y
3. Translate back to original reference

2D Scaling Algorithm:

1. Start
2. Input original coordinates (x,y) , scaling factors s_x and s_y , and fixed point (x_f, y_f)
3. Apply scaling transformation using the above formulas
4. Update object coordinates to (x',y')
5. Stop

Program for 2D Scaling

```
#include <stdio.h>
#include <graphics.h>

void drawTriangle(int x1, int y1, int x2, int y2, int x3, int y3) {
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
}

void scale(int *x, int *y, float sx, float sy, int xf, int yf) {
    *x = xf + round((*x - xf) * sx);
    *y = yf + round((*y - yf) * sy);
}
```

```

}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    int x1 = 100, y1 = 100;
    int x2 = 200, y2 = 100;
    int x3 = 150, y3 = 200;

    // Draw original triangle
    drawTriangle(x1, y1, x2, y2, x3, y3);

    float sx = 1.5, sy = 1.5; // scaling factors
    int xf = 150, yf = 150;   // fixed point (center)

    // Scale triangle
    scale(&x1, &y1, sx, sy, xf, yf);
    scale(&x2, &y2, sx, sy, xf, yf);
    scale(&x3, &y3, sx, sy, xf, yf);

    // Draw scaled triangle
    drawTriangle(x1, y1, x2, y2, x3, y3);

    getch();
    closegraph();
    return 0;
}

```

Implementation / Output

Output Description:

The program displays a triangle before and after scaling. The original triangle is drawn in one color, and the scaled triangle is drawn in another. The output clearly labels:

Object: Original triangle

Image: Scaled triangle

Quadrant: Both triangles are shown within the first quadrant, scaled about a specified fixed point.

(Attach a printout of your graphics output here, with clear labels.)

Conclusion

This lab successfully demonstrated the implementation of 2D scaling in Computer Graphics. By applying scaling factors s_x and s_y about a fixed point, the triangle was enlarged or shrunk accurately while preserving its shape proportions. The exercise reinforced the mathematical and matrix foundations of scaling and its practical application in graphical transformations. The output clearly showed the change in size, confirming the correctness of the implemented algorithm.

References

<https://drive.google.com/file/d/127EGfaWylCqVLYn2QnpizLJs0ktia9nT/view?usp=sharing>